

DATA-STRUCTURES & ALGORITHMS

with

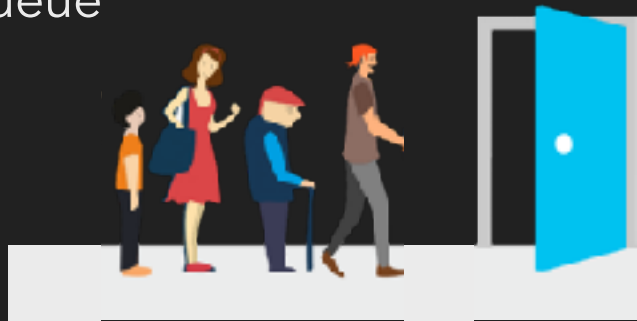
Manoj Prabhakaran

Lecture 7

Queue

Queue

- What is a queue?
 - An item can be added only to the back of the queue
 - Can remove only the front-most item
 - **First-In First-Out (FIFO)**
- Why are queues useful in programs?
 - Handling events (key presses, mouse clicks, network packets arriving at a router, ...) in the order in which they happened
 - Maintaining a window of recent items: Add new item, remove old item
 - A forgetful stack (without any popping) will work too!
 - Systematic searches (We said that about stacks too!)
 - ...



Queue

- Can implement using a linked list, and have $O(1)$ enqueue and dequeue operations
- A quick implementation using a vector?
 - Say a fixed-capacity queue?
 - Not immediate! Vector offers $O(1)$ operations at only one end
- Will come back to this soon. For now, let's assume we have a fixed-capacity queue
- Can we implement a full-fledged queue given a fixed-capacity queue?
 - Yes! Similar to what we did for stacks!

A Queue as a List of Bounded Queues

```
struct queueBlock; // TODO
struct queueNode { queueBlock block; queueNode* behind = nullptr; }
struct queueList { // a list of queueNodes
    queueNode *head = nullptr, *tail = nullptr;
    bool empty() { return head==nullptr; } // queue empty iff list empty
    void enqueue(char x) { // will enqueue into tail node
        if(tail==nullptr) head = tail = new queueNode; // if empty list
        if(!tail->block.try_enqueue(x)) { // may fail if tail is full
            queueNode* tmp = new queueNode; tail->behind = tmp; tail = tmp;
            tail->block.try_enqueue(x); // try again after adding a new node
        }
    }
    bool dequeue(char& x) {
        if (empty()) return false;
        head->block.dequeue(x); // if not empty, dequeue from front block
        if(head->block.empty()) { // if it is empty after dequeue, delete it
            queueNode* tmp = head->behind; delete head; head = tmp;
            if(head==nullptr) tail = nullptr;
        }
        return true;
    }
};

bool front(char& x) {
    if(empty()) return false;
    return head->block.front(x);
}
```

An Application

- A simple moving average is used in many situations to monitor real-time performance of systems
 - E.g., “telemetry data” measure system load at any point. If the load has been high for sometime, need to allocate more resources
 - Don’t want to allocate resources just when the telemetry data spikes (the spike may pass)
 - Given telemetry data x_i , use $a_t = \frac{s_t}{N}$ where $s_t = x_t + x_{t-1} + \dots x_{t-N+1}$
 - Can update from s_{t-1} to s_t in $O(1)$ time: $s_t = s_{t-1} + x_t - x_{t-N}$
 - But need to remember x_{t-N} from time $t - N$ till time t

Simple Moving Average

```
#include <queue>
```

```
struct Averager {  
    const size_t N = 50; // window (size_t, as returned by std::queue::size())  
    std::queue<double> data;  
    double sum = 0;  
    void ingest(float x) {  
        data.push(x); sum += x;  
        if(data.size() > N) { sum -= data.front(); data.pop(); }  
    }  
    bool getAverage(double& a) {  
        if(data.size() < N) return false;  
        a = sum/N; return true;  
    }  
};
```

std::queue uses push(), pop()
instead of enqueue(), dequeue()

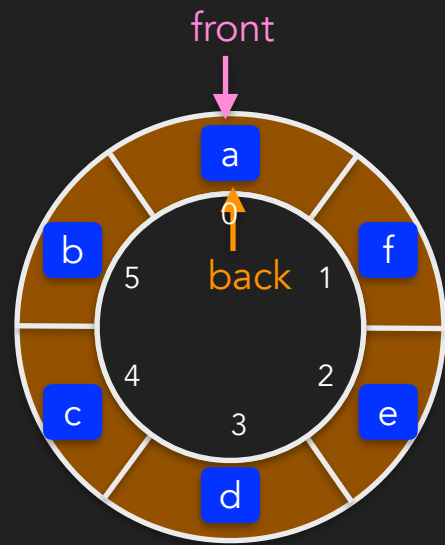
pop() doesn't return the dequeued value;
use front() to read it before popping.

Fixed-Capacity Queue

- Our problem: Vector offers $O(1)$ operations at only one end, but queue needs operations at both ends
- We already solved a very similar problem
 - Recall Forgetful Stack
 - Had to push/pop items at one end
 - But also had to forget from the other end!
 - We implemented it using a circular array (array indexed modulo the size of the array)
 - Same idea works for fixed-capacity queues!

A Fixed-Capacity Queue

```
struct queueBlock {  
    const int N = 512; char Q[N]; // fixed capacity Q  
    int front = 0, back = 0; // Can be any index with front==back  
    bool is_empty = true; // front==back can mean 0 or 1 item  
    void dec(int& i) { i = (i==0 ? N-1:i-1); } // decrement mod N  
    bool full() {return (back==0 && front==N-1)|| (back==front+1);}   
    bool try_enqueue(char x) {  
        if(full()) return false;  
        if(is_empty) is_empty = false; else dec(back);  
        Q[back] = x; return true;  
    }  
    bool dequeue(char& x) {  
        if(is_empty) return false;  
        x = Q[front];  
        if(front==back) is_empty = true; // had one item, now empty  
        else dec(front); // if not emptying, decrement front  
        return true;  
    }  
    bool front(char& x) { x = Q[front]; return !is_empty; }  
};
```



is_empty 

Summary: Implementing a Queue

- Fixed-Capacity queue from circular arrays
- Full-fledged queue from a linked list of fixed-capacity queues
 - $O(1)$ inserts and deletes
- The constant hidden in $O(1)$ can be large if every insertion/deletion needs to add/delete a node in the list
 - Recall: Same issue in our stackList implementation last time
 - Mitigated by removing nodes slightly lazily, so that adding/removing nodes will be required only once every B operations (or more), B being queueBlock capacity
 - Same fix applies here too

Another Application: Systematic Exploration

- In the game of minesweeper, some cells have mines under them. Other cells either have a neighbouring cell that is mined (dangerous cells) or not (safe cells).
- When you click on a safe cell, it opens up an “island” of safe cells surrounded by a sea of dangerous cells
- How do you find the cells belonging to this island?
- High-level idea: Systematically explore starting from the clicked cells and moving to its safe neighbours
 - Concretely: Maintain a queue of safe cells encountered so far, and checkout the neighbours of each cell in the queue

Another Application: Systematic Exploration

- If the clicked cell is safe, start by enqueueing it
- Repeat while queue not empty:
 - Dequeue a cell
 - Enqueue all its neighbours that are safe but were not already enqueued
- Claim: Cells enqueued are exactly those in the island (i.e., connected to the starting node by a "path" of safe cells)

a	b	c	d		
g					
m	n	o	p		
s	t	u	v	w	x
y	z	A	B	C	D
E	F	G	H	I	J

Can you prove this?

Hint: Use induction on the length of the path

Later

[f] → [e k l] → [q r e k] → [j q r e] → [j q r] → [j q] → [j] → [i] → [h] → []

Another Application: Systematic Exploration

- What if we had used a stack or any other container?
- *If the clicked cell is safe, insert it into the container*
- *Repeat while container not empty:*
 - *Remove a cell from the container*
 - *Insert all its neighbours that are safe but were not already inserted*
- Still works! But the order of discovery can be different
 - With queue, cells closer to the start are discovered before the further off ones

a	b	c	d		
g					
m	n	o	p		
s	t	u	v	w	x
y	z	A	B	C	D
E	F	G	H	I	J

[f] → [l k e] → [l k j] → [l k q i] → [l k q h] → [l k q] → [l k r] → [l k] → [l] → []

Summary

- A queue provides only enqueue (push) and dequeue (pop) operations
- No random access and hence no need to maintain items contiguously
- Needs $O(1)$ operations at both ends: Can't use `std::vector`
- We saw two implementations with $O(1)$ worst-case push/pop
 - **queueBlock**: A queue with a pre-determined capacity, using a circular array
 - **queueList**: An (uncapped) queue using a list of queueBlocks
- Applications discussed: Processing items in the order they arrived, calculating Simple Moving Averages, systematic exploration.